

Evaluación Parcial N° 5

Encargo | Estudiante

Subdirección de Diseño
Curricular e Instruccional
2025

MALETA DIDÁCTICA DuocUC

Evaluación Parcial N° 5

Encargo | Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación	Semana inicio EFT
DSY1107	Desarrollo Cloud Native I	5 horas pedagógicas	12%	

1. Instrucciones generales

Descripción

- Esta etapa busca evaluar la capacidad del estudiante para integrar sistemas de streaming y analítica.
- Se observará el uso correcto de tópicos, la publicación y consumo de eventos, la justificación del diseño, y cómo Kafka complementa la arquitectura existente sin afectar las integraciones previas con RabbitMQ.
- La distribución de los porcentajes en esta evaluación es la siguiente:

Evaluación	Tipo de situación evaluativa	Distribución de porcentajes	Individual/Grupal
Evaluación Parcial N°5 (12%)	Encargo	100%	Parejas
Total		100%	

- El encargo corresponde al código de los componentes frontend y backend del sistema, entregado vía GitHub.
- El tiempo asignado para desarrollar esta evaluación es de 2 semanas para la elaboración del encargo y de la presentación.
- Las/los estudiantes inician el trabajo del encargo en el Taller de Proyectos (Taite 7), pero deben finalizarlo junto a presentación en su tiempo de trabajo personal.

Instrucciones específicas de la Evaluación:

- Los componentes desarrollados, frontend y backend se entregan mediante el envío a el/la docente del código fuente.
- El código de todos los componentes del backend debe compilar, seguir buenas prácticas y responder a pruebas básicas.
- El código del frontend debe estar completo, modular, sin errores de compilación y con vistas funcionales.
- La integración del backend con la base de datos cloud debe estar configurada correctamente mediante entidades, repositorios y propiedades de conexión.
- El backend debe incluir filtros que validen el JWT recibido desde el IDaaS para autorizar las peticiones.
- El frontend debe implementar el flujo de login con IDaaS y utilizar el JWT en las llamadas al backend.
- Para ambos contextos, front y back, solo debe subirse a los repositorios lo que corresponda a la tecnología utilizada, configurando los archivos .gitignore necesarios para lograr esto.
- Los microservicios deben incluir en su código productores y consumidores que se conecten a colas RabbitMQ.
- Los microservicios deben manejar mensajes no entregados enviándolos a DLQ y registrándolos en logs.
- El microservicio administrador de RabbitMQ debe permitir la creación y eliminación de colas, exchanges y bindings.
- Los microservicios deben incluir en su código productores y consumidores que se conecten a tópicos Kafka.
- El microservicio administrador de Kafka debe permitir la creación, modificación y eliminación de tópicos y configuraciones.
- El proyecto debe incluir un archivo docker-compose funcional para levantar un clúster de RabbitMQ.
- El proyecto debe incluir un archivo docker-compose funcional para levantar una arquitectura Kafka con Zookeeper y brokers.
- El proyecto debe incluir un archivo docker-compose funcional para levantar Kafka-ui y permitir la administración de brokers y tópicos

Los aspectos formales son:

- Formato: el backend debe corresponder a varios microservicios construidos en Java con Spring Boot, el frontend debe ser un componente Angular. Ambos tipos de componentes deben ser entregados como enlaces a Github.
- Medio de entrega: Copiar los enlaces de los repositorios de Github a AVA y enviar copia al correo del docente, en el plazo de entrega establecido.

Los materiales, herramientas o insumos que se requieren para realizar esta evaluación:

- Materiales y contenidos académicos proporcionados durante el curso, incluyendo especificaciones técnicas, análisis previos y guías de referencia.
- Computadores disponibles en taller de proyectos (TAITE 7)

2. Pauta de Evaluación

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	**100%**	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Buen desempeño	**80%**	Demuestra un alto desempeño del indicador, presentando pequeñas omisiones, dificultades y/o errores.
Desempeño aceptable	**60%**	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	**30%**	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	**0%**	Presenta ausencia o incorrecto desempeño.

Indicador de Evaluación	Categorías de Respuesta					Ponderación Indicador de Evaluación
	Muy buen desempeño 100%	**Buen desempeño 80%**	**Desempeño aceptable 60%**	**Desempeño incipiente 30%**	**Desempeño no logrado 0%**	
El código del microservicio incluye la configuración completa del cliente Kafka, con conexión a los brokers levantados en Docker Compose y propiedades que permiten publicar y consumir mensajes desde los tópicos con sus particiones y réplicas definidas.	La configuración del cliente Kafka está completa y correctamente definida incluyendo bootstrap servers serializers consumer groups y propiedades avanzadas. Permite	La configuración es funcional y contiene todos los elementos principales aunque falta algún parámetro secundario o mejora menor.	La configuración está presente pero incompleta o dispersa lo que dificulta su mantenimiento o provoca inconsistencias en el consumo o la publicación.	La configuración existe pero es incorrecta insuficiente o no permite la comunicación estable con los brokers.	No existe configuración del cliente Kafka en el código de los microservicios.	12

	publicar y consumir mensajes desde ambos tópicos sin cambios adicionales.					
El código declara productores y consumidores que interactúan con los dos tópicos configurados, utilizando estructuras tipadas como DTO y validando que los microservicios puedan escribir y leer mensajes de manera coherente con la arquitectura Kafka del proyecto.	Productores y consumidores están completamente implementados usando DTO estructurados y lógica clara. Interactúan correctamente con ambos tópicos siguiendo la arquitectura definida.	Productores y consumidores funcionan de forma adecuada aunque presentan oportunidades menores de mejora en la estructura o validación.	Los productores o consumidores están implementados de forma parcial o no interactúan consistentemente con ambos tópicos.	El código incluye productores o consumidores pero funcionan de manera incorrecta o inestable.	No se observa implementación funcional de productores o consumidores.	12
El código del microservicio administrador incluye endpoints o clases que permiten crear, listar o mantener componentes de Kafka como tópicos y configuraciones básicas, demostrando capacidad de gestión sobre la plataforma de streaming.	El microservicio administrador implementa operaciones completas para crear listar y administrar tópicos y configuraciones básicas funcionando como una capa de gestión sólida.	Implementa la mayoría de las operaciones necesarias aunque alguna función secundaria está incompleta.	El microservicio ofrece solo un subconjunto de las operaciones y requiere acciones externas para tareas de administración.	Las capacidades de administración son mínimas o no confiables.	No implementa endpoints o clases para administrar componentes de Kafka.	6
El código de los microservicios contiene productores Kafka correctamente implementados, enviando eventos a los tópicos configurados con uso de DTO bien estructurados y serializers adecuados para los tipos de datos manejados.	Los productores están correctamente implementados con DTO claros serializers adecuados y lógica alineada	Los productores funcionan de manera correcta aunque existe algún detalle menor por mejorar en	Los productores existen pero presentan inconsistencias o cubren solo parte del flujo esperado.	Los productores están implementados pero no son funcionales o presentan errores frecuentes.	No existen productores Kafka en el código.	14

	al diseño del sistema.	validación o estructura.				
El código implementa consumidores Kafka que procesan eventos de forma coherente con los flujos de negocio, utilizando @KafkaListener u otra API equivalente y manteniendo la lógica de procesamiento separada de la infraestructura de mensajería.	Los consumidores están completamente implementados usando @KafkaListener o equivalente y procesan los eventos respetando los flujos de negocio con clara separación entre lógica de dominio y mensajería.	Los consumidores funcionan correctamente aunque se observan oportunidades menores de mejora en separación de responsabilidades.	Los consumidores están implementados pero de forma parcial o con uso inconsistente de los tópicos.	Los consumidores existen pero no procesan eventos correctamente o la implementación es inestable.	No existen consumidores Kafka en el código.	14
El código demuestra manejo adecuado de errores durante el procesamiento de eventos, utilizando estrategias como logging, validación de payloads o control de reintentos que aseguran que los mensajes defectuosos o inesperados no interrumpan el funcionamiento del sistema.	El manejo de errores está completamente implementado incluyendo validaciones logging contextual y control de reintentos evitando efectos adversos en el sistema.	El manejo de errores es correcto aunque falta cubrir algunos escenarios excepcionales.	Existe un manejo básico de errores pero no es suficiente para asegurar estabilidad ante situaciones complejas.	El manejo de errores es mínimo y deja al sistema vulnerable ante fallas.	No existe manejo de errores en el procesamiento de eventos.	12
El código o archivos de configuración contienen parámetros de administración de tópicos tales como niveles de retención, cleanup policy, número de particiones y factor de replicación ajustados a los requerimientos de los distintos flujos de datos.	Los parámetros de administración están correctamente configurados y alineados con los requerimientos	La configuración incluye la mayoría de los parámetros necesarios con pequeños ajustes pendientes.	Algunos parámetros están configurados pero faltan elementos clave para la administración completa del tópico.	La configuración existe pero es incorrecta o insuficiente para los requerimientos del sistema.	No se observan configuraciones de administración de tópicos.	12

	de cada flujo de datos.					
El código demuestra la configuración y uso de consumer groups adecuados para balancear carga o asegurar procesamiento resiliente permitiendo que distintos microservicios compartan o dividan responsabilidades según el caso de uso.	Los consumer groups están correctamente configurados y permiten balanceo resiliencia y distribución según las necesidades del sistema.	Los consumer groups están implementados de forma adecuada aunque con menos detalle estratégico en su configuración.	Existen consumer groups pero no están bien alineados al caso de uso o están configurados parcialmente.	Los consumer groups existen pero no funcionan correctamente o no influyen en el procesamiento.	No se emplean consumer groups en el sistema.	12
El código del microservicio administrador o scripts de infraestructura incluyen operaciones para crear, actualizar o inspeccionar tópicos y grupos de consumidores mostrando capacidad de gestionar la plataforma Kafka de manera controlada y replicable.	El microservicio o scripts implementan operaciones completas y replicables para administrar tópicos y consumer groups demostrando dominio total sobre la gestión de Kafka.	Implementa la mayoría de las operaciones aunque con algún detalle pendiente para casos más avanzados.	Incluye algunas operaciones de administración pero faltan funcionalidades clave para la gestión completa.	La administración es mínima y no demuestra un uso real de las herramientas.	No existen herramientas o servicios para administrar Kafka en el sistema.	6
					Total	**100%**